



Sistemas Paralelos - 2026

Trabajo Práctico N° 6 *Open Specifications for Multi-Processing (OpenMP)*



Objetivos de la práctica: Implementar programas paralelos en OpenMP.

Bibliografía:

- Especificación de OpenMP: <https://www.openmp.org/specifications/>
- An Introduction to Parallel Programming. 1st Edition. Peter Pacheco. Morgan Kaufmann. 2011. ISBN 9780123742605.

PARTE I: INTRODUCCIÓN

NOTA: no utilizar worksharing en esta parte (recién se utiliza en la PARTE II).

- 1) Completar el siguiente programa usando la directiva: `#pragma omp parallel num_threads(10)`. Mantener la declaración de la variable `id` antes de la región paralela.

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int id;
    {
        id = omp_get_thread_num();
        usleep(500);
        printf("Soy el hilo %d de %d hilos...\n", id, omp_get_num_threads());
    }
    return 0;
}
```

Programa 1: paralelización básica

- Ejecutar el programa varias veces. ¿Cuántas veces se presenta cada hilo?
- ¿En cada ejecución se obtiene el mismo resultado? ¿Por qué?
- Agregar al final del `pragma` la cláusula `private(id)`. ¿Cuántas veces saluda cada hilo? ¿Por qué?
- Modificar el programa de tal manera que la única salida del programa sea la cantidad de threads de la región paralela. El programa debe hacer una sola impresión del valor y el `printf` debe estar ubicado dentro de la región paralela.
- Modificar la versión de tal manera que el `printf` esté ubicado después de la región paralela. ¿Por qué la impresión del número de threads aparece siempre luego de que todos los hilos se presentan?

2) Realizar una variante del programa anterior donde el número de threads se establece desde fuera del programa, a través de la variable de entorno correspondiente.

3) Considere el siguiente programa:

```
#include <stdio.h>
#include <unistd.h>
#include <omp.h>

int inc(int var){
    usleep(1000);
    return var + 1;
}

int main(int argc, char **argv){
    int var;

    var = 10;
    { //región paralela
        var = inc(var);
        printf("var (in): %d\n", var);
    }
    printf("var (out): %d\n", var);
    return 0;
}
```

Programa 2: tipo de variables

Crear versiones paralelas con 4 hilos, donde:

- Se pase la variable como `private` a la región paralela
- Se pase la variable como `private` a la región paralela
- Se pase la variable como `shared` a la región paralela
- Se pase la variable como `shared` a la región paralela, pero la llamada a la función `inc()` sea `critical`.

En cada caso, ejecutar y razonar sobre los valores que la variable `var` toma dentro y fuera de la región paralela.

¿Para qué sirve la directiva `critical` empleada en la última versión? ¿Qué consecuencia tiene utilizarla?

4) Suponga que tiene el siguiente programa:

```
#include <stdio.h>
#include <omp.h>

int main(){
    int id;
    #pragma omp parallel num_threads(10) private(id)
    {
        id = omp_get_thread_num();
        printf("%d: Hola!\n", id);
        printf("%d: Adios!\n", id);
    }
    return 0;
}
```

Programa a: sincronización básica

- Compile y ejecute el programa varias veces. ¿Se mezclan los mensajes de distintos threads? ¿Por qué?
- Modifique la función principal de la siguiente forma:

```
int main(){
    int id;
    #pragma omp parallel num_threads(10) private(id)
    {
        id = omp_get_thread_num();
        printf("%d: Hola!\n", id);
    }
    #pragma omp parallel num_threads(10) private(id)
    {
        id = omp_get_thread_num();
        printf("%d: Adios!\n", id);
    }
}
```

Programa b: sincronización básica

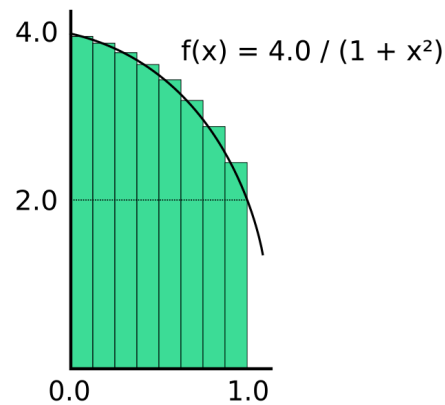
- ¿En qué se diferencia la salida de esta versión con la anterior? ¿Por qué?
- Implemente una versión con una única región paralela que genere el mismo efecto en la salida, utilizando la construcción *barrier*.

5) Es posible calcular el valor de π con el siguiente método. Matemáticamente, sabemos que:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

y que es posible calcular la integral de manera aproximada como una suma de rectángulos:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$



en donde cada rectángulo tiene un ancho Δx y altura $f(x_i)$ considerando que x_i es el punto medio de la base del rectángulo.

El siguiente programa serie calcula el número π utilizando este método.

```
#include <stdio.h>
void main () {
    long num_steps = 800000000;
    double step;
    int i;
    double x, pi, sum = 0.0;
    step = 1.0/(double) num_steps;
    for (i = 0; i < num_steps; i++){
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }
    pi = step * sum;
    printf("%f\n",pi);
}
```

- a) Crear una versión paralela del programa usando la construcción:
#pragma omp parallel

Utilizar threads dinámicos. No utilizar secciones críticas ni la cláusula reduction. En reemplazo, utilizar un arreglo en donde cada thread debe ir acumulando su propio valor de sum. Cada thread, por cada iteración del bucle que deba procesar, actualizará el elemento del arreglo que le corresponda de acuerdo a su número de thread. Al finalizar la región paralela, realizar la suma de ese arreglo.

Prestar especial atención a las variables compartidas y privadas.

Para construir el programa se necesitará utilizar las siguientes funciones:

- `omp_get_max_threads()` retorna el valor de la ICV `nthreads-var` que sirve para dimensionar el arreglo que almacena los valores sum.
- `omp_get_thread_num()`
- `omp_get_num_threads()`

- b) Para el programa del inciso (a), obtener tiempos de ejecución y speedups para 2, 4, 6 y 8 threads.

- c) Modificar el programa del inciso (a), reemplazando su declaración del arreglo por la siguiente (considerando que su arreglo se llama `sum`).

```
double sum[nthreads][8];
```

Deberá ajustar el resto del código del programa. Por ejemplo, donde dice `sum[i]` deberá decir `sum[i][0]`.

Contrastar el rendimiento de esta versión con el de la versión anterior. ¿A qué se debe la mejora?

- d) Modificar el programa del ejercicio (a) de tal manera que, en vez de un arreglo, se utilicen variables privadas a cada thread y, mediante una sección crítica (construcción crítica), se actualice la variable `sum`. Contrastar el rendimiento de esta versión con el de las versiones anteriores. ¿Qué pasaría si la sección crítica se coloca dentro del bucle?
- e) Modificar el programa del inciso (d), utilizando, para actualizar la variable `sum`, la cláusula `reduction` en reemplazo de la sección crítica. Contrastar el rendimiento de esta versión con el de las versiones anteriores.

PARTE II: WORKSHARING-FOR

6) Suponga que tiene el siguiente programa:

```
#include <stdio.h>
#include <unistd.h>

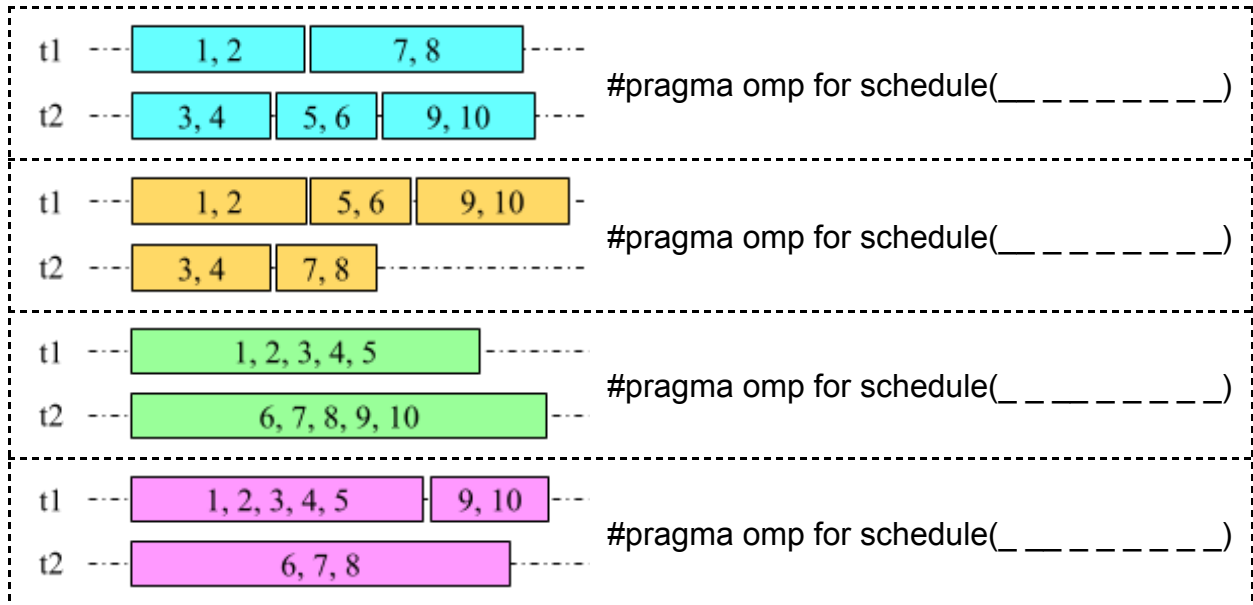
void trabajo(int id){
    printf("%d: Trabajo muy duro...\n", id);
    sleep(1);
    printf("%d: como un esclavo!\n", id);
    sleep(1);
}

int main(){
    trabajo(0);
    trabajo(1);
    trabajo(2);
    trabajo(3);
    trabajo(4);
    trabajo(5);
    trabajo(6);
    trabajo(7);
    return 0;
}
```

Programa 3: worksharing for

- Escriba las instrucciones necesarias para medir el tiempo de ejecución. ¿Cuánto tiempo demora?
- Modifique el código, utilizando una construcción *for*, para acelerarlo usando 2, 4 y 8 hilos.

7) Para cada uno de los gráficos, que muestran las asignaciones de iteraciones a los threads $t1$ y $t2$ durante una cierta ejecución, completar la cláusula `schedule` con una planificación que puede producir esa ejecución.



8) Reemplazar las líneas punteadas por la planificación (`static/dynamic/guided`) que usted crea que es más adecuada para cada programa. Luego, ejecutar cada programa para comprobar la validez de sus hipótesis. Finalmente, dar conclusiones generales sobre las planificaciones.

a) El cómputo que se realiza por cada iteración es siempre el mismo. Para este caso, es apropiado utilizar la planificación: _____.

```
#include <omp.h>
int main()
{
    int i, j;
    #pragma omp parallel
    {
        int a = 0;
        #pragma omp for schedule(_____) private(j)
        for (i = 0; i < 100000000; i++) {
            for(j = 0; j < 100; j++)
                a = a + j/4;
        }
    }
}
```

b) El cómputo que se realiza por cada iteración varía según si es índice par o impar. Para este caso, es apropiado utilizar la planificación: _____.

```

#include <omp.h>
int main()
{
    int i, j;
    #pragma omp parallel
    {
        int a = 0;
        #pragma omp for schedule(_ _ _ _) private(j)
        for (i = 0; i < 1000000000; i++) {
            if (i % 2 == 0)
                for(j = 0; j < 100; j++)
                    a = a + j/4;
        }
    }
}

```

- c) El cómputo que se realiza por cada iteración es aleatorio (función rand). Para este caso, es apropiado utilizar la planificación: _____ .

```

#include <omp.h>
#include <stdlib.h>
int main()
{
    int i, j;
    #pragma omp parallel
    {
        int a = 0;
        #pragma omp for schedule(_ _ _ _) private(j)
        for (i = 0; i < 1000000000; i++) {
            if (rand() % 2 == 0)
                for(j = 0; j < 100; j++)
                    a = a + j/4;
        }
    }
}

```

- d) El cómputo que se realiza por cada iteración es igual, sin embargo, algunos threads llegan, a la construcción for, un rato antes que el resto. Para este caso, es apropiado utilizar la planificación: _____ .

```

#include <omp.h>
int main()
{
    int i, j;
    #pragma omp parallel
    {
        int a = 0;
        int t = omp_get_thread_num();
        if (t > 10) sleep(1);
        #pragma omp for schedule(_ _ _ _) private(j)
        for (i = 0; i < 1000000000; i++) {
            for(j = 0; j < 100; j++)
                a = a + j/4;
        }
    }
}

```

```
}  
}
```

9) Resolver el ejercicio 5 utilizando una construcción for. Contrastar esta versión con el de las versiones anteriores en rendimiento y la cantidad de modificaciones respecto al código serie.

10) Según las condiciones de Bernstein, indica el tipo de dependencias de datos existente entre las distintas iteraciones en los casos que se presentan a continuación. Justifica si se puede eliminar o no esa dependencia de datos. En caso de ser posible la eliminación, escribir una versión paralela.

a)

```
int i, j, A[1000];  
j = 5;  
for (i = 0; i < 1000; i++) {  
    j += 2;  
    A[i] = 3 * j * j * j + 2 * j * j + 10 * j + 2;  
}
```

b)

```
int i, x[1000];  
for (i = 1; i < 999; i++) {  
    x[i+1] = x[i] + x[i-1];  
}
```

c) Ayuda: tiene solución!

```
int i, x[1000], y[1001], z[1000];  
for (i = N - 2; i >= 0; i--) {  
    x[i] = x[i] + y[i+1];  
    y[i] = y[i] + z[i];  
}
```

11) Escribir un programa paralelo que utilice una construcción for para calcular la cantidad de números primos dentro de un rango de números naturales. Se espera una solución con carga balanceada; para verificar que la carga esté balanceada, imprimir el tiempo que cada thread tarda en ejecutar el worksharing loop (se deberá eliminar la barrera implícita al final del worksharing).

Ejemplo: si el rango es [10, 31], los números primos en ese rango son: 11, 13, 17, 19, 23, 29 y 31. Así, la salida del programa debe ser 7, porque hay 7 números primos entre 10 y 31.

Utilizar la siguiente función para determinar si un número es o no primo:


```
int primo(int n) {  
    int i,res,mitad;  
    i = 2;  
    res = 1;  
    mitad = n / 2;  
    while(res != 0 && i <= mitad) {  
        res = n % i;  
        i++;  
    }  
    return res;  
}
```

12) Dado el siguiente código serie, escribir un código paralelo OpenMP equivalente que utilice las construcciones parallel y for. No se debe modificar la estructura del algoritmo. No paralelizar la inicialización. Poner especial atención en las dependencias de datos. Considerar el uso de la cláusula nowait en los worksharing for. Compilar con la opción -lm (para la librería math) y -O3 (nivel 3 de optimización).

Ejecutando con un nodo de uso exclusivo del cluster: ¿Con cuántos threads se alcanza el menor tiempo de ejecución? .

Para esa cantidad de threads, indicar:

Tiempo de ejecución: .

Speedup: .

Código fuente del programa serie:

```

#include <stdio.h>
#include <math.h>
#define N 1000
#define PASOS 1000

int main()
{
    double A[N][N], b[N], x[N], x2[N];
    double max=-1.0, q=0, s, e;
    int i, j, k, n=N, pasos=PASOS;

    //inicialización
    for (i=0;i<n;i++)
        b[i] = 0.5;
    for (i=0;i<n;i++)
        x[i] = 0.5+(double)i/n;
    for (i=0;i<n;i++)
        for (j=0;j<n;j++)
            A[i][j] = 0.6*(sin(i+j)+1)/2;
    //fin inicialización

    for (k=0;k<pasos;k++) {
        q=1;
        for (i=0;i<n;i++) {
            s = b[i];
            for (j=0;j<n;j++) {
                e = sin(A[i][j] * x[j]);
                s+= e * e * e + 2 * e * e + 5 * e;
            }
            x2[i] = s;
            q *= sin(s);
        }
        for (i=0;i<n;i++)
            x[i] = x2[i];
        for (i=0;i<n/2;i++)
            b[i] = b[i] + 0.00002;
        if (max<q) {
            max = q;
            printf("max: %.10e\n", max);
        }
    }
    printf("max final: %.10e\n", max);
    return max;
}

```

PARTE III: WORKSHARING-SECTIONS

13) Suponga que tiene el siguiente programa:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <omp.h>
void tarea_uno() {
    sleep(2);
}
void tarea_dos() {
    sleep(3);
}
void tarea_tres() {
    sleep(2);
}
int main () {
    double tiempoIni, tiempoFin;
    tiempoIni = omp_get_wtime();
    printf("Ejecutando tarea 1\n");
    tarea_uno();
    printf("Ejecutando tarea 2\n");
    tarea_dos();
    printf("Ejecutando tarea 3\n");
    tarea_tres();
    tiempoFin = omp_get_wtime();
    printf("Tiempo: %f segundos\n", tiempoFin - tiempoIni);
    return 0;
}
```

- a) Crear una región paralela y utilizar una construcción sections para ejecutar las tres tareas (independientes) en paralelo. Mostrar el tiempo de ejecución total de la región paralela.
- b) Modificar la salida para que cada tarea indique en qué thread fue ejecutada. Mostrar el tiempo de ejecución con 2, 3 y 4 threads. Justificar los resultados.

PARTE IV: TAREAS EXPLÍCITAS

14) Ejecutar el siguiente programa y responder:

- a) ¿Puede quedar impreso en la terminal un chau antes que un hola?
 - b) ¿Cuántas tareas creó cada thread?
 - c) ¿La cantidad de tareas que ejecuta cada thread es la misma que las que crea?
- Modificar el programa para imprimir el identificador del thread que la ejecuta.

```
#include <omp.h>
#include <stdio.h>
int main()
{
    #pragma omp parallel num_threads (2)
    {
        int x;
        int id = omp_get_thread_num();
        for (x = 0; x < 10; x++) {
            #pragma omp task
            printf("hola %d\n", id);
        }
        #pragma omp taskwait
        printf("chau %d\n", id);
    }
}
```

15) Se desea implementar la suma vectorial $C = A + B$, para vectores de 2.000.000.000 de elementos.

- a) Completar el siguiente código para obtener una versión secuencial recursiva:

```
#define N 2000000000
void vector_add(int *A, int *B, int *C, int n) {
    int i;
    for (i = 0; i < n; i++)
        C[i] = A[i] + B[i];
}

void rec_vector_add(int *A, int *B, int *C, int n) {
    // si es el caso base, llamar a vector_add
    // sino, hacer la suma recursiva sobre la primera mitad de los
    // arreglos, y luego la suma recursiva sobre la segunda mitad de los
    // arreglos.
}

int main() {
    rec_vector_add(a, b, c, N);
}
```

- b) Implementar una versión paralela utilizando la construcción **task** y el paradigma divide y vencerás. Evaluar el rendimiento del programa y explicar los resultados

utilizando un caso base de:

- i) 10 elementos
- ii) 1.000.000 elementos

16) Implementar el ejercicio 5 utilizando la construcción task y el paradigma divide y vencerás. El programa debe permitir definir un tamaño mínimo de tarea.

Definir (para el valor predeterminado de pasos) cuál es el tamaño de tarea adecuado para lograr una carga balanceada mientras se reduce la sobrecarga por gestión de la paralelización.